

Convolução 1D *from scratch*

Bernardo Mendes Rebello¹ and Lucas Pinto Avelar¹

¹Instituto de Computação, Universidade Federal Fluminense

Resumo

Este trabalho detalha a implementação de uma Rede Neural Convolucional unidimensional (CNN 1D) desenvolvida *from scratch*, utilizando exclusivamente a biblioteca NumPy [1] para a manipulação de tensores. O objetivo central do estudo reside na dedução matemática e na implementação vetorizada dos algoritmos de *Forward Propagation* e, principalmente, do *Backpropagation* em camadas convolucionais e de *Max Pooling*. Para assegurar a corretude da implementação, foram realizadas comparações com uma implementação equivalente no *framework* PyTorch [2]. A arquitetura proposta foi aplicada à tarefa de reconhecimento de atividades humanas (HAR) utilizando dados de acelerômetro do conjunto WISDM [3]. O artigo demonstra as operações matriciais necessárias para o treinamento de redes profundas sem o auxílio de diferenciação automática.

Introdução

Este trabalho apresenta o desenvolvimento de uma CNN 1D *from scratch* (do zero), utilizando exclusivamente a biblioteca NumPy para manipulação de tensores. O objetivo central não é apenas a reprodução funcional de uma rede neural, mas a dedução e implementação vetorizada dos algoritmos de *Forward Propagation* e, principalmente, do *Backpropagation* através de camadas convolucionais, de *pooling* e densas.

A arquitetura desenvolvida foi projetada para a classificação de atividades humanas baseada em dados de sensores (acelerômetro), utilizando o conjunto de dados **WISDM** (Wireless Sensor Data Mining). O fluxo da rede implementada é construído da seguinte forma:

$$\textit{Entrada} \rightarrow \textit{Conv1D} \rightarrow \textit{ReLU} \rightarrow \textit{MaxPool} \rightarrow \textit{Flatten} \rightarrow \textit{FC} \rightarrow \textit{Sigmoid}$$

Para validar a implementação e garantir a corretude, os resultados numéricos de cada etapa foram comparados com uma implementação equivalente construída no *framework* PyTorch.

O trabalho está organizado de forma a detalhar a implementação modular de cada camada. Inicialmente, discutimos a redução de dimensionalidade via *MaxPooling*. Em seguida, aprofundamos na matemática da *Convolução 1D*, detalhando as operações tensoriais para o cálculo de gradientes

de pesos e entradas. Por fim, apresentamos outras camadas da arquitetura como a *Flatten*, a *Fully Connected* e a camada de função de ativação, demonstrando como o fluxo de gradientes é tratado nesse processo.

Vale ressaltar que as funções de custo (*Loss Function*) e otimização foram adaptadas de outras implementações já existentes.

Camada de Redução: MaxPooling

Para a redução de dimensionalidade (downsampling), foi implementada a camada de *Max Pooling*. Nesta abordagem, optou-se por fixar o *stride* (passo) ao tamanho da janela (*kernel size*) e não utilizar dilatação para essa camada, resultando em janelas não sobrepostas. Assim, o tamanho da janela é o único parâmetro que pode ser alterado na camada.

O fluxo da camada é dividido em duas etapas principais:

Forward Pass

A camada recebe um tensor de entrada e aplica uma janela deslizante para cada canal de forma independente. Para cada janela, é selecionado apenas o valor máximo, descartando o restante. A saída será um tensor de dimensão reduzida na proporção do tamanho da janela. Durante o processo do forward, a cada janela, uma matriz que cumpre a função de máscara binária (*mask*) é armazenada. Esta máscara possui as mesmas dimensões da entrada e contém o valor 1 na posição onde o máximo foi encontrado e 0 nas demais.

Backward Pass

Como a camada de Max Pooling não possui parâmetros treináveis (pesos ou *bias*), sua função no backpropagation é apenas redirecionar o gradiente vindo da camada posterior para as posições corretas na camada anterior. O processo ocorre em dois passos, como ilustrado na imagem:

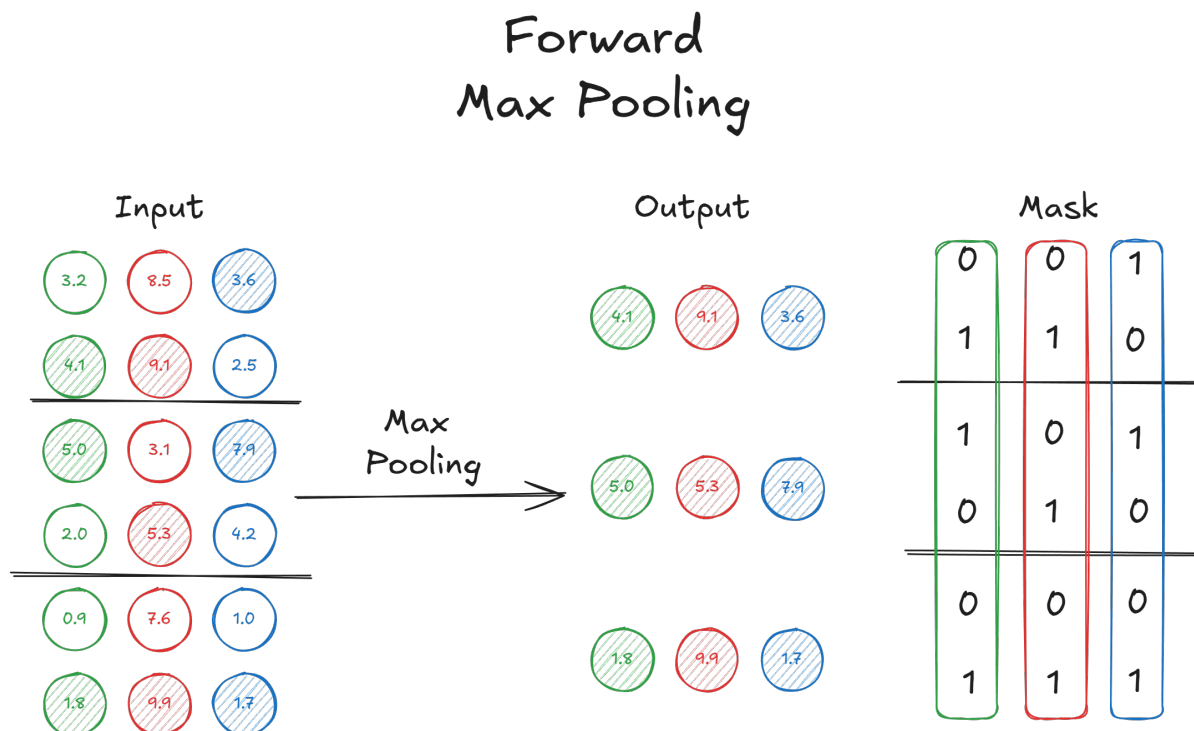


Figura 1: Max Pooling Forward Pass

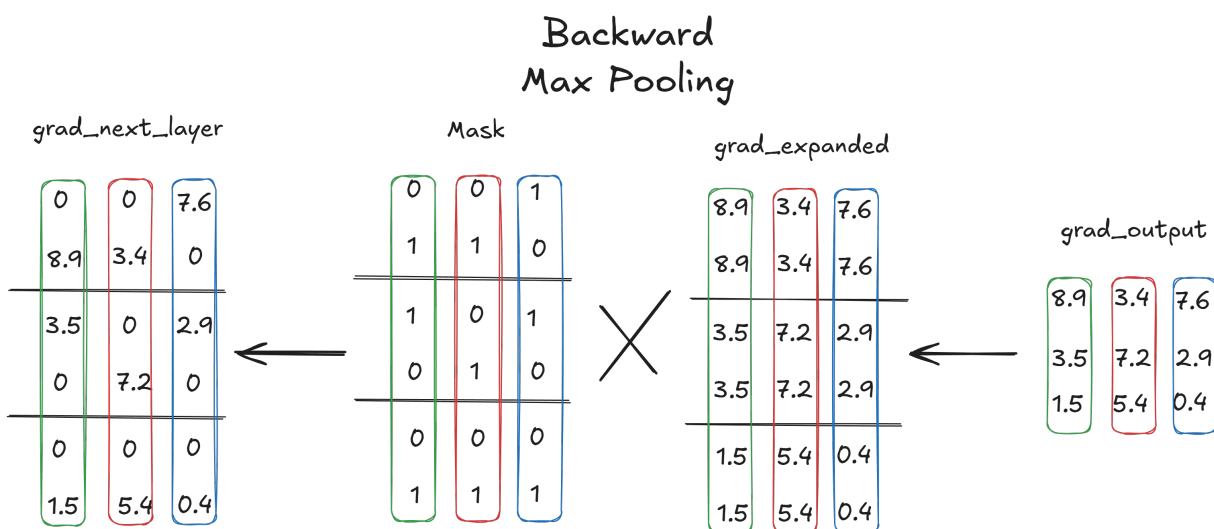


Figura 2: Max Pooling Backward Pass

1. **Expansão do Gradiente:** O gradiente recebido (que possui dimensão reduzida) é expandido através da repetição de seus valores para se adequar ao tamanho da janela original.
2. **Filtragem:** O gradiente expandido é multiplicado (produto ponto a ponto) pela máscara

salva no forward.

Isso garante que o erro flua apenas através dos neurônios que geraram o erro na etapa da propagação direta, atribuindo gradiente zero para as ativações que não contribuíram para a saída.

A Camada Convolutiva 1D: Forward Pass

A camada convolutiva unidimensional (Conv1D) se destaca como o principal componente dessa implementação. Diferentemente das redes densas (fully connected), que tratam a entrada de forma global, a camada convolutiva preserva a estrutura espacial e temporal dos dados através do compartilhamento de pesos. A seguir, definimos os componentes fundamentais que estruturam as operações de *forward* e *backward* pass ilustradas nas próximas seções.

Estrutura dos Dados de Entrada

A entrada da camada é interpretada como uma sequência contínua, que possui duas dimensões principais:

- A dimensão do comprimento dos dados
- A dimensão dos canais de entrada.

No contexto visual dos diagramas que serão apresentados, os canais são representados pelas diferentes cores (verde, vermelho, azul).

O Conceito de Janela e Filtros

Para processar essa entrada, a camada utiliza dois conceitos-chaves que definem nossa notação:

- **Janelas Deslizantes (J):** A convolução 1D funciona deslizando uma "janela" de tamanho fixo ao longo do comprimento da entrada.
- **Coleção de Filtros (f):** A rede aprende por meio de um conjunto de filtros (kernels). Cada filtro é uma matriz de pesos independente, em que o número de colunas é igual ao número de canais da entrada. O objetivo de cada filtro é se especializar na detecção de um padrão específico.

Algoritmo de Forward Pass

O funcionamento do algoritmo durante a etapa (*Forward Pass*) foi representado visualmente no diagrama da Figura 3. O processo ilustra a transformação dos dados de entrada nas saídas através de operações de convolução.

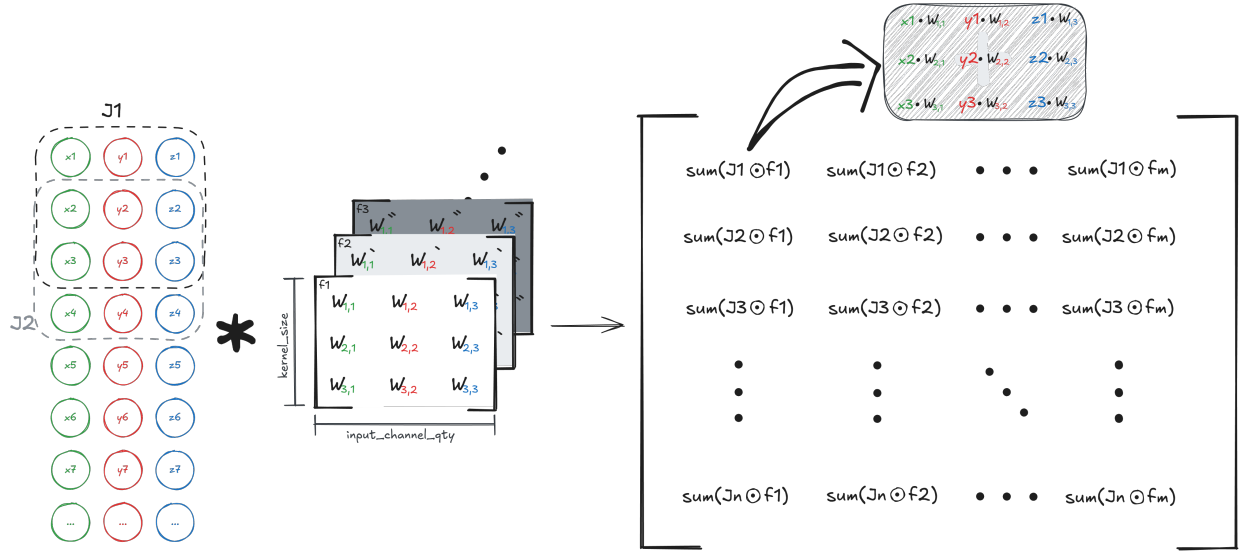


Figura 3: Representação visual do Forward Pass: das janelas deslizantes (J) à matriz de saída (O).

Conforme detalhado na imagem, o procedimento pode ser descrito em três etapas principais:

1. Segmentação em Janelas (J): O lado esquerdo do diagrama apresenta a entrada sendo fatiada em janelas deslizantes, denotadas por $J_1, J_2, \dots, J_n$. Cada janela captura um recorte da entrada contendo todos os canais de entrada (representados pelas cores verde, vermelho e azul). O deslocamento vertical das janelas ilustra o conceito de *stride*. Para o uso de *padding*s, são adicionadas linhas de zeros nas extremidades das entradas.

2. Aplicação dos Filtros (f): A parte central demonstra a coleção de filtros treináveis da rede, identificados como $f_1, f_2, \dots, f_m$. Cada filtro possui uma matriz de pesos W (ex: $W_{1,1}, W_{1,2}, \dots$) que será aplicada em cada janela da entrada. Para cada par de uma janela com um filtro, realiza-se o somatório da multiplicação ponto a ponto, denotado na imagem por $\text{sum}(J \odot f)$. Isso significa que cada ponto da entrada é ponderado pelo seu respectivo peso no kernel e somado para gerar um único valor. Em caso de dilatação, são adicionadas linhas de zeros entre as linhas do filtro, para representar o comportamento da dilatação diretamente no filtro.

3. Construção da Matriz de Saída: O lado direito do diagrama apresenta a estrutura de dados resultante. A saída é organizada em uma matriz onde:

- As **linhas** correspondem a cada janela ($J_1 \dots J_n$) processadas sequencialmente).
- As **colunas** correspondem aos diferentes filtros aplicados ($f_1 \dots f_m$).

Desta forma, cada elemento da matriz final contém o resultado de um filtro específico sobre uma janela específica, servindo de entrada para as camadas subsequentes da rede neural.

Convolução 1D: Backward Pass

Gradiente em relação à entrada (dX)

O objetivo é propagar o erro em relação a entrada para a camada anterior. A Figura 4 ilustra a lógica deste cálculo, destacando a **sobreposição** dos gradientes.

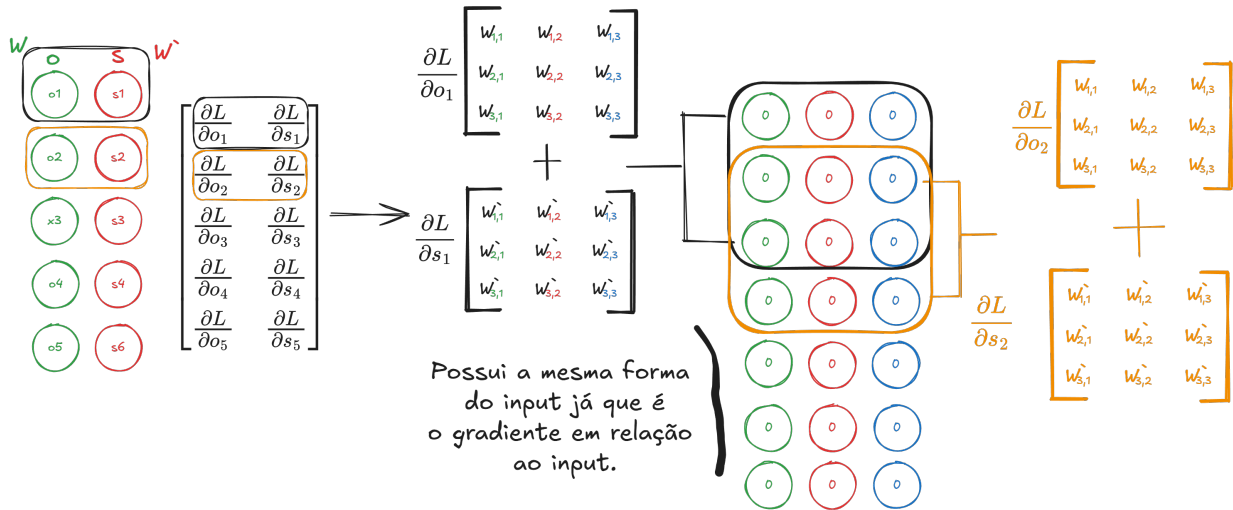


Figura 4: Cálculo de dX : A reconstrução do gradiente de entrada através da soma das contribuições sobrepostas das janelas de saída.

Diferente do *forward pass*, onde a dimensão é reduzida, aqui o objetivo é restaurar a dimensionalidade original da entrada. A imagem detalha este processo da seguinte forma:

1. Aplicação dos Pesos (W e W'):

Cria-se uma matriz de zeros com a mesma forma (*shape*) do input. O diagrama mostra que os pesos originais dos filtros (W para o primeiro filtro e W' para o segundo) são aplicados sobre essa matriz de gradientes.

- Cada peso contribui para distribuir o erro de volta para os elementos da entrada que ele multiplicou durante o *forward pass*.
- A operação visualizada é análoga a deslizar o kernel em cada linha do output sobre os gradientes preenchidos, assim como ilustrado na região superior esquerda da figura.

2. Soma das Contribuições dos Filtros:

O lado esquerdo e direito desta equação demonstram que o gradiente final da entrada é a soma das contribuições de todos os filtros ponderada pelo erro recebido.

$$\frac{\partial L}{\partial X} = (\text{Contribuição do Filtro } W) + (\text{Contribuição do Filtro } W')$$

Isso garante que, se um dado de entrada foi utilizado por múltiplos filtros para gerar diferentes

elementos da saída, o erro proveniente de todas essas características seja acumulado corretamente no respectivo ponto de entrada.

Gradiente em relação aos pesos (dW)

O cálculo dos gradientes dos pesos (dW) foi implementado através de operações de álgebra linear, com objetivo de delegar o máximo de processamento ao numpy. O processo consiste em calcular uma linha de pesos do kernel de cada vez, utilizando o fatiamento da entrada com a matriz de gradientes de saída. A Figura 5 ilustra o procedimento específico para o cálculo da **2ª linha de pesos** de todos os filtros (W , W' , W'').

Calcular

2ª linha de
todos os filtros

$$\begin{bmatrix} x1 & x2 & x3 & x4 & x5 & x6 & x7 & \dots \\ y1 & y2 & y3 & y4 & y5 & y6 & y7 & \dots \\ z1 & z2 & z3 & z4 & z5 & z6 & z7 & \dots \end{bmatrix} \times \frac{\partial L}{\partial O} = \begin{bmatrix} \frac{\partial L}{\partial O_{1,1}} & \frac{\partial L}{\partial O_{1,2}} & \dots & \frac{\partial L}{\partial O_{1,m}} \\ \frac{\partial L}{\partial O_{2,1}} & \frac{\partial L}{\partial O_{2,2}} & \dots & \frac{\partial L}{\partial O_{2,m}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial O_{n,1}} & \frac{\partial L}{\partial O_{n,2}} & \dots & \frac{\partial L}{\partial O_{n,m}} \end{bmatrix}$$

$$= \begin{bmatrix} \frac{\partial L}{\partial W_{2,1}} & \frac{\partial L}{\partial W'_{2,1}} & \frac{\partial L}{\partial W''_{2,1}} \\ \frac{\partial L}{\partial W_{2,2}} & \frac{\partial L}{\partial W'_{2,2}} & \frac{\partial L}{\partial W''_{2,2}} \\ \frac{\partial L}{\partial W_{2,3}} & \frac{\partial L}{\partial W'_{2,3}} & \frac{\partial L}{\partial W''_{2,3}} \end{bmatrix}$$

Filtros

Figura 5: Cálculo matricial para a 2ª linha dos kernels: Multiplicação entre a fatia deslocada da entrada (transposta) e o gradiente da saída.

O algoritmo segue a lógica abaixo:

1. Seleção da Região da Entrada: Para calcular os gradientes referentes à k -ésima linha do kernel (no exemplo da imagem, a 2ª linha), selecionamos uma "fatia" da entrada original transposta deslocada por $k - 1$ passos (para esse caso específico em que a dilatação é 1). A ideia das "fatias" é alcançar todos os elementos que $W_{2,i}$, $W'_{2,i}$ e $W''_{2,i}$

- Como, na imagem, estamos calculando a **2ª linha** dos filtros, ignoramos a primeira coluna e pegamos a sequência a partir da segunda.

2. Produto Matricial: A operação central é a multiplicação de duas matrizes:

$$\text{Resultado} = (\text{Entrada}_{\text{slice}}) \times \left(\frac{\partial L}{\partial O} \right)$$

Onde:

- **Entrada_{slice}:** Matriz de dimensões $(3 \times N)$, contendo todos os canais de entrada ao longo da janela válida.
- **Gradiente $\frac{\partial L}{\partial O}$:** Matriz de dimensões $(N \times m)$, contendo o erro propagado para cada filtro.

3. Interpretação da Matriz Resultante: O resultado desta multiplicação é uma matriz $(3 \times m)$ que contém exatamente os gradientes para a linha específica do kernel que estamos calculando, mas para todos os filtros simultaneamente. Conforme mostrado na equação da imagem:

$$dW_{\text{row}=2} = \begin{bmatrix} \frac{\partial L}{\partial W_{2,1}} & \frac{\partial L}{\partial W'_{2,1}} & \frac{\partial L}{\partial W''_{2,1}} \\ \frac{\partial L}{\partial W_{2,2}} & \frac{\partial L}{\partial W'_{2,2}} & \frac{\partial L}{\partial W''_{2,2}} \\ \frac{\partial L}{\partial W_{2,3}} & \frac{\partial L}{\partial W'_{2,3}} & \frac{\partial L}{\partial W''_{2,3}} \end{bmatrix} \quad (1)$$

Como não utilizamos esse padrão de posição para os pesos, essa matriz é transposta quando for utilizada pelo otimizador. Para que os pesos sejam atualizados corretamente

Gradiente em relação ao Viés (dB)

Como o mesmo valor de viés B_k é somado a cada passo da janela deslizante do filtro k , a regra da cadeia determina que seu gradiente total seja a soma das derivadas parciais em relação a todas as posições de saída. Dado a matriz de gradientes da saída $\frac{\partial L}{\partial O}$, o cálculo do gradiente para o viés do k -ésimo filtro é dado por:

$$\frac{\partial L}{\partial B_k} = \sum_{i=1}^{N_{\text{output}}} \frac{\partial L}{\partial O_{i,k}} \quad (2)$$

Em uma matriz, esta operação corresponde à soma das colunas da matriz de gradiente, o vetor de gradiente do viés dB será:

$$dB = \left[\sum_{i=1}^N \frac{\partial L}{\partial O_{i,1}} \quad \sum_{i=1}^N \frac{\partial L}{\partial O_{i,2}} \quad \cdots \quad \sum_{i=1}^N \frac{\partial L}{\partial O_{i,m}} \right] \quad (3)$$

Onde dB é um vetor de gradientes de viés de tamanho (M_{filtros}) .

Camadas de Adaptação e Classificação

Após a etapa convolucional, também foram implementadas as camadas restantes da nossa arquitetura: Flatten, Fully Connected e Ativação. A separação destas operações em camadas distintas isola as responsabilidades matemáticas e simplifica o fluxo de dados tanto durante a etapa *forward* e principalmente na etapa do *backpropagation*.

Camada Flatten (Achatamento)

A camada *Flatten* atua como uma interface de adaptação dimensional. As camadas convolucionais produzem como saída uma matriz onde as dimensões espaciais são preservadas. No entanto, uma camada totalmente conectada subsequente espera receber um vetor unidimensional como entrada.

Forward Pass: A operação consiste em "esticar" a matriz de entrada X_{conv} de dimensões $(N \times C)$ em um único vetor V de dimensão $(1 \times K)$, onde $K = N \cdot C$, N é o comprimento da entrada e C é a quantidade de canais da entrada.

Backward Pass: Como esta camada não possui pesos treináveis, apenas são reorganizados os gradientes recebidos da camada densa de volta para o formato matricial original, para que possam ser propagados para a convolução. Isso é realizado, salvando o formato original e reestruturando o dado para esse formato.

Camada Fully Connected (Linear)

Como não estamos utilizando processamento em *batches*, as operações simplificam-se para álgebra linear vetorial direta.

Forward Pass: Dado o vetor de entrada achatado X e a matriz de pesos W :

$$Z = X \cdot W + B \quad (4)$$

Backward Pass: Durante a retropropagação, calculamos como o erro varia em relação aos pesos, ao viés e à entrada. Sem a dimensão de batch, o gradiente do viés (dB) será idêntico ao gradiente recebido ($\frac{\partial L}{\partial Z}$), pois não há acumulação de erros de múltiplos exemplos.

$$\text{Gradiente dos Pesos: } \frac{\partial L}{\partial W} = X^T \cdot \frac{\partial L}{\partial Z} \quad (5)$$

$$\text{Gradiente do Bias: } \frac{\partial L}{\partial B} = \frac{\partial L}{\partial Z} \quad (6)$$

$$\text{Gradiente da Entrada: } \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} \cdot W^T \quad (7)$$

Camada de Ativação

A camada de ativação opera elemento a elemento sobre o vetor de saída Z , introduzindo a não-linearidade necessária para a rede neural.

Forward Pass: Aplicação direta da função σ (ex: ReLU, Sigmoid):

$$A = \sigma(Z) \quad (8)$$

Backward Pass: A propagação do gradiente ocorre através da multiplicação elemento a elemento ($\odot$) pelo gradiente local da função de ativação (σ').

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \sigma'(Z) \quad (9)$$

Resultados e Validação

Para validar os algoritmos de *Backpropagation* e das operações de convolução implementadas via NumPy, foi realizado um experimento comparativo contra o *framework* PyTorch.

Arquitetura da Rede

Para isolar a validação das operações matemáticas, ambas as implementações utilizaram a mesma arquitetura, projetada para processar janelas temporais de acelerômetro (3 eixos). A configuração detalhada dos hiperparâmetros e dimensões de cada camada encontra-se na Tabela 1.

Tabela 1: Arquitetura detalhada da CNN 1D utilizada no experimento

Camada	Configuração	Detalhes
Entrada	$Channels = 3$	Eixos X, Y, Z
Convolução 1D	$Filters = 4, Kernel = 3$	Sem padding, Stride=1
Ativação	ReLU	-
Pooling	MaxPool 1D	$Kernel = 2, Stride = 2$
Flatten	-	Vetorização de (4×99)
Fully Connected	$In = 396, Out = 1$	Neurônio único de saída
Saída	Sigmoid	Classificação Binária

Configuração do Experimento e Diferenças de Implementação

O experimento seguiu a metodologia de **Inicialização Sincronizada**: no primeiro instante, os pesos aleatórios gerados pelo PyTorch foram copiados para a implementação NumPy, garantindo o mesmo ponto de partida na superfície de erro.

Um ponto crucial de distinção entre as implementações reside na manipulação dos dados durante o treinamento:

Para tornar os métodos comparáveis, a taxa de aprendizado da implementação NumPy foi ajustada proporcionalmente ao tamanho do conjunto de dados ($\alpha_{numpy} = \alpha_{torch}/N$), aproximando a magnitude das atualizações estocásticas ao comportamento do gradiente em lote.

Análise de Convergência

A Figura 6 apresenta a evolução da função de custo e da acurácia ao longo de 15 épocas.

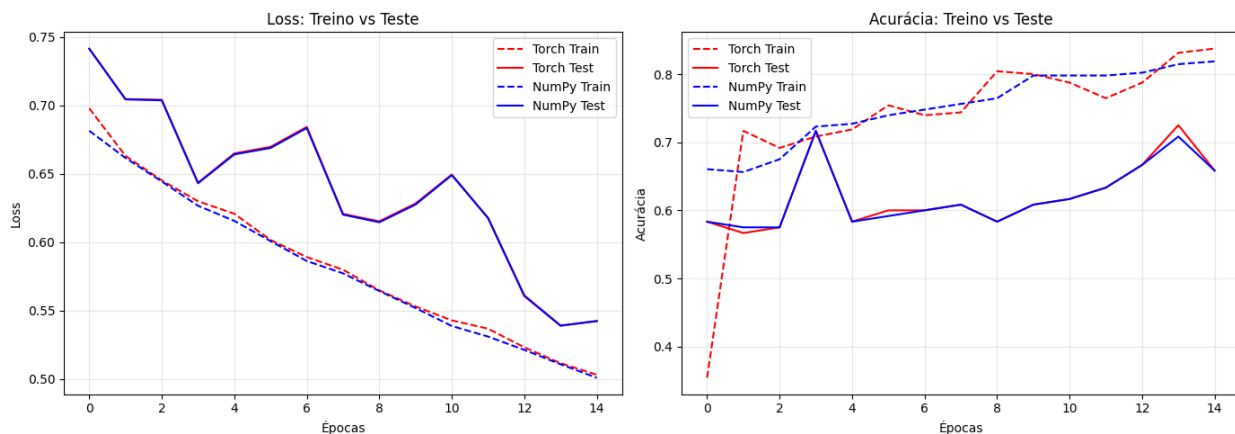


Figura 6: Comparação de convergência entre PyTorch e NumPy. À esquerda, a minimização da Loss Function. À direita, a evolução da Acurácia.

Apesar da diferença estrutural no processamento (Batch vs. Amostra), observa-se uma sobreposição consistente das curvas de teste e treino. A convergência das duas valida que a dedução dos gradientes (dW , dB , dX) na implementação *from scratch* está matematicamente correta, reproduzindo com fidelidade a dinâmica de aprendizado de uma biblioteca de alto desempenho.

Referências

- [1] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [2] A. Paszke, S. Gross, F. Massa, *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, Curran Associates, Inc., 2019.
- [3] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, “Activity recognition using cell phone accelerometers,” *ACM SIGKDD Explorations Newsletter*, vol. 12, no. 2, pp. 74–82, 2011.